BÁO CÁO THỰC HÀNH MINI PROJECT 04

TEAM GIT WORKFLOW SIMULATION

Người báo cáo: Liamz07

1. Tổng quan và Mục tiêu dự án:

Mini Project 04 tập trung vào việc thiết kế, cấu hình và vận hành thực tế một quy trình quản lý mã nguồn chuyên nghiệp theo mô hình Gitflow dành cho đội ngũ phát triển phần mềm nhiều thành viên.

Mục tiêu cốt lõi của dự án không chỉ dừng lại ở các thao tác lệnh Git cơ bản, mà đi sâu vào Quản trị kho chứa (Repository Governance): thiết lập các hàng rào kỹ thuật để ngăn chặn lỗi đẩy mã trực tiếp, ép buộc kiểm duyệt chéo (Code Review), giải quyết xung đột (Merge Conflict) khi làm việc song song và xử lý sự cố khẩn cấp trên môi trường sản phẩm (hotfix/*).

2. Kiến trúc đội ngũ & Mô hình phân nhánh:

PHẦN 1: PHÂN VAI ĐỘI NGŨ PHÁT TRIỂN:

Hệ thống mô phỏng sự phối hợp đồng bộ giữa 3 lập trình viên với vai trò và trách nhiệm rõ ràng:

Thành viên	Vai trò trong nhóm	Nhiệm vụ thực thi trong mô phỏng
Developer A	Tech Lead / Maintainer	Khởi tạo cấu trúc kho chứa, thiết lập quy tắc bảo vệ nhánh (Branch Protection), đảm nhận phát triển tính năng xác thực (feature/auth) và thực hiện Code Review / Merge.
Developer B	Backend / Feature Developer	Phát triển tính năng thanh toán (feature/payment), chủ động phối hợp xử lý xung đột mã nguồn (Merge Conflict) với Developer A.
Developer C	DevOps / Production Support	Theo dõi hệ thống chạy thật (main), xử lý các lỗi nghiêm trọng phát sinh đột xuất thông qua nhánh vá lỗi khẩn cấp (hotfix/critical-bug).

PHẦN 2: HỆ THỐNG PHÂN CẤP NHÁNH:

Kho chứa được tổ chức thành 4 tầng nhánh chuẩn mực:

- **main (Production Branch):** Nhánh cao nhất, chỉ chứa mã nguồn đã được kiểm thử kỹ lưỡng và sẵn sàng chạy trên môi trường thực tế. Tuyệt đối cấm đẩy mã trực tiếp (direct push).
- **develop (Integration Branch):** Nhánh phát triển trung tâm, nơi hội tụ tất cả các tính năng mới từ các lập trình viên trước khi đóng gói phát hành lên main.
- **feature/* (Feature Branches):** Các nhánh chức năng tách ra từ develop (ví dụ: feature/auth, feature/payment) để từng thành viên lập trình độc lập.

- **hotfix/* (Hotfix Branches):** Các nhánh vá lỗi khẩn cấp tách trực tiếp từ main (ví dụ: hotfix/security-patch) để khắc phục sự cố nghiêm trọng và hợp nhất ngược lại vào cả main lẫn develop.

develop [(tách nhánh) →] feature/ [(PR + Review) →] develop [(Release PR) →] main [(sự cố khẩn) →] hotfix/* [(PR + Review) →] main & develop*

3. Thiết lập chính sách quản trị:

Để mô phỏng đúng tiêu chuẩn doanh nghiệp trên GitHub (Settings → Branches → Add branch protection rule), 4 chính sách kiểm soát chất lượng đã được áp dụng cho hai nhánh cốt lõi là main và develop:

- **Pull Request bắt buộc:**
  - *Kích hoạt tùy chọn "Require a pull request before merging".*
  - *Mọi thay đổi muốn đưa vào main hoặc develop đều không thể dùng lệnh git push trực tiếp mà bắt buộc phải đi qua luồng tạo Pull Request.*
- **Bảo vệ nhánh:**
  - *Chặn hoàn toàn thao tác xóa nhánh (Do not allow bypassing the above settings / chặn xóa nhánh chính).*
  - *Khóa quyền ghi đè lịch sử (Block force pushes — vô hiệu hóa lệnh git push --force) để bảo toàn tính toàn vẹn lịch sử commit.*
- **Kiểm duyệt chéo:**
  - *Kích hoạt "Require approvals" (yêu cầu tối thiểu 1 lượt phê duyệt từ thành viên khác trước khi nút Merge được mở khóa).*
  - *Kích hoạt "Dismiss stale pull request approvals when new commits are pushed" (tự động hủy trạng thái đã duyệt nếu tác giả đẩy thêm code mới sau khi review, đảm bảo mọi dòng code cuối cùng đều được kiểm tra).*
- **Chính sách hợp nhất:**
  - *Kích hoạt "Require conversation resolution before merging":* Bắt buộc tất cả các bình luận góp ý (Review comments) phải được đánh dấu Resolved mới cho phép hợp nhất.
  - *Chuẩn hóa chiến lược Merge:* Sử dụng Squash and merge đối với các nhánh feature/* vào develop (giúp lịch sử develop sạch gọn, mỗi tính năng là 1 commit duy nhất) và Create a merge commit khi phát hành từ develop lên main.

4. Kịch bản mô phỏng vận hành chi tiết:

GIAI ĐOẠN 1: KHỞI TẠO DỰ ÁN & KIỂM THỬ BẢO VỆ NHÁNH

- **Khởi tạo kho chứa và đẩy cấu trúc cơ sở:**

```
mkdir team-git-workflow && cd team-git-workflow
git init
echo "# Enterprise Web Application" > README.md
echo "PORT=3000" > config.env
git add . && git commit -m "chore: initial project structure"
git branch -M main
git remote add origin <repository-url>
git push -u origin main

# Tạo nhánh develop từ main
git checkout -b develop
git push -u origin develop
```

- **Kiểm chứng cơ chế Bảo vệ nhánh:**
  - *Developer A thử sửa tệp README.md ngay trên nhánh main và thực hiện git push origin main.*
  - *Kết quả:* GitHub từ chối ngay lập tức với thông báo lỗi: remote: error: GH006: Protected branch update failed for refs/heads/main (Changes must be made through a pull request). Cơ chế bảo vệ hoạt động chính xác.

GIAI ĐOẠN 2: PHÁT TRIỂN TÍNH NĂNG SONG SONG & XỬ LÝ XUNG ĐỘT (DEVELOPER A & DEVELOPER B)

Bước 2.1: Developer A hoàn thành tính năng feature/auth

```
git checkout develop
git checkout -b feature/auth
echo "AUTH_SECRET=super_secret_key" >> config.env
echo "function login() { return 'authenticated'; }" > auth.js
git add . && git commit -m "feat(auth): implement user login logic"
git push -u origin feature/auth
```

- Developer A mở Pull Request #1 (feature/auth → develop).
- Developer B vào thẻ Files changed, kiểm tra mã nguồn, nhấn Approve, sau đó PR #1 được Squash and merge thành công vào develop.

Bước 2.2: Developer B phát triển feature/payment và gặp Merge Conflict

Trước đó, Developer B đã tách nhánh feature/payment từ bản develop cũ (khi chưa có code của Developer A) và cũng chỉnh sửa đúng dòng cuối của tệp config.env:

```
# Trên máy của Developer B
git checkout -b feature/payment
echo "PAYMENT_GATEWAY=stripe_live" >> config.env
echo "function pay() { return 'success'; }" > payment.js
git add . && git commit -m "feat(payment): integrate payment gateway"
git push -u origin feature/payment
```

- Khi Developer B mở Pull Request #2 (feature/payment → develop), GitHub cảnh báo: This branch has conflicts that must be resolved tại tệp config.env.
- **Quy trình giải quyết xung đột chuẩn của Developer B:**

```
# Kéo cập nhật mới nhất của nhánh develop về máy cục bộ
git fetch origin
git merge origin/develop

# Mở tệp config.env, xóa thẻ đánh dấu <<<<<<<, =======, >>>>>>> và giữ lại cả 2 cấu hình:
# PORT=3000
# AUTH_SECRET=super_secret_key
# PAYMENT_GATEWAY=stripe_live

git add config.env
git commit -m "chore: resolve merge conflict in config.env with develop"
git push origin feature/payment
```

- **Kết quả:** Cảnh báo xung đột trên PR #2 biến mất. Developer A thực hiện Code Review, để lại nhận xét góp ý xử lý ngoại lệ, Developer B cập nhật và đánh dấu Resolve conversation, sau đó PR #2 được phê duyệt và hợp nhất vào develop.

GIAI ĐOẠN 3: ĐÓNG GÓI PHÁT HÀNH & XỬ LÝ SỰ CỐ KHẨN CẤP HOTFIX/* (DEVELOPER C)

- **Phát hành phiên bản v1.0.0 lên main:**
  - *Developer A tạo Pull Request #3 (develop → main), được nhóm kiểm duyệt và hợp nhất chính thức lên môi trường Production (main).*
- **Phát hiện sự cố và kích hoạt nhánh hotfix/port-fix (Developer C):**
  - *Giả lập sự cố cấu hình cổng mạng trên main bị sai (PORT=3000 cần đổi gấp sang PORT=8080 để khớp với Reverse Proxy Nginx).*
  - *Developer C tách nhánh nóng trực tiếp từ main:*

```
git checkout main
git pull origin main
git checkout -b hotfix/port-fix

# Sửa cổng trong config.env thành PORT=8080
sed -i 's/PORT=3000/PORT=8080/g' config.env
git add config.env
git commit -m "fix(config): update production port to 8080"
git push -u origin hotfix/port-fix
```

- **Hợp nhất kép để đồng bộ hệ thống:**
  - *PR #4 (hotfix/port-fix → main):* Developer A phê duyệt khẩn cấp và merge ngay để khôi phục dịch vụ Production.
  - *PR #5 (hotfix/port-fix → develop):* Hợp nhất bản vá ngược lại vào develop nhằm đảm bảo các tính năng phát triển sau này không làm tái diễn lỗi cũ, sau đó xóa nhánh hotfix/port-fix.

5. Tổng kết và bài học kinh nghiệm:

Qua quá trình triển khai và vận hành Mini Project 04 — Team Git Workflow Simulation, nhóm thực hiện đã đúc kết được những giá trị cốt lõi trong quy trình kỹ nghệ phần mềm hiện đại:

- **Kỷ luật phân nhánh:** Việc tách bạch rõ ràng vai trò của main, develop, feature/* và hotfix/* giúp đội ngũ nhiều lập trình viên có thể làm việc song song trên cùng một kho mã nguồn mà không giẫm chân lên nhau.
- **Sức mạnh của Repository Governance:** Các thiết lập Branch Protection, Pull Request, Code Review và Merge Policy đóng vai trò như một hệ thống miễn dịch tự động, ngăn chặn triệt để tình trạng code chưa kiểm thử lọt vào nhánh chính và nâng cao chất lượng mã nguồn thông qua phản biện chéo.
- **Tư duy xử lý tình huống thực tế:** Làm chủ quy trình đồng bộ nhánh để giải quyết Merge Conflict chủ động từ máy cục bộ trước khi merge, cũng như quy trình Dual Merge cho nhánh hotfix/* để giữ cho toàn bộ hệ thống luôn nhất quán.

HẾT